Object Oriented Programming (CSEG1044)
Unit 01 – Lecture 02 Notes
Java Overview + Program Structure + Features

April 1, 2026

Contents

1 Lecture Snapshot **2**

2 Lecture Overview **2**

3 Prerequisite Bridge **2**

4 Learning Outcomes **2**

5 Suggested Reading Order **3**

6 Key Terms (Quick) **3**

7 Detailed Notes **3**

7.1 Java ecosystem: JDK vs JRE vs JVM . . . 3

7.2 Bytecode and the compile/run flow . . . 4

7.3 Basic program structure: class + `main` . . . 4

7.4 Compile-time vs runtime errors . . . 4

7.5 Features of Java (what they mean in practice) . . . 4

8 Running Project Connection **5**

9 Common Mistakes and Confusions **5**

10 Worked Example: Compile & Run + Observe Errors **5**

11 Demo / Observation Task **6**

12 Practice Check (with brief solutions) **6**

13 Self-Study Practice (no solutions) **6**

14 Further Study **7**

15 Exit Questions (with sample answers) **7**

1 Lecture Snapshot

- **Unit focus:** Introduction to OOPs and Java Fundamentals
- **Main new ideas:** Java ecosystem: JDK vs JRE vs JVM, Bytecode and the compile/run flow, Basic program structure: class + `main`
- **Course thread:** Use Java Overview + Program Structure + Features to make the first text-based version of Campus Resource Manager easier to reason about before the full object model arrives.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

2 Lecture Overview

This lecture makes you comfortable with the Java "toolchain" and the minimum you need to compile and run Java programs. By the end, you should clearly understand:

- what the **JDK**, **JRE**, and **JVM** mean,
- how **bytecode** enables Java to run on different platforms,
- the difference between **compile-time errors** and **runtime errors**.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

3 Prerequisite Bridge

Before reading the new material in depth, do a fast recall of the older ideas listed below.

- **OOP vocabulary**: The previous lecture introduced objects, classes, and modelling; this lecture connects them to the Java platform. Main reference: `Unit 01 / Lecture 01 slides/notes`.
- **Problem to program flow**: Once the model is clear, the next question is how Java compiles and runs it. Main reference: `Unit 01 / Lecture 01 slides/notes`.

This lecture only revisits those ideas briefly so that the main explanation can stay focused on the new concept sequence.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

4 Learning Outcomes

After this lecture, you should be able to:

- Explain JDK vs JRE vs JVM and what bytecode is.

- Compile and run a Java program; recognize compile-time vs runtime errors.
- List key features of Java and their practical meaning.

5 Suggested Reading Order

1. Read the **Prerequisite Bridge** first and confirm each recalled idea.
2. Study the central explanation in this order: *Java ecosystem: JDK vs JRE vs JVM → Bytecode and the compile/run flow → Basic program structure: class + `main` → Compile-time vs runtime errors → Features of Java (what they mean in practice).*
3. Trace the worked example line by line before checking short answers.
4. Attempt the practice section without looking at the solution immediately.
5. Finish with the exit questions and further-study suggestions to confirm retention.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

6 Key Terms (Quick)

- **JDK (Java Development Kit)**: tools for development (compiler `javac`, docs, etc.).
- **JRE (Java Runtime Environment)**: runtime needed to run Java apps (JVM + libraries).
- **JVM (Java Virtual Machine)**: executes Java bytecode (`.class`).
- **Bytecode**: platform-neutral instructions produced by `javac`.
- **Classpath**: where the JVM looks for compiled classes and libraries.
- **Compile-time error**: detected by the compiler before running.
- **Runtime error/exception**: occurs while the program is executing.

7 Detailed Notes

7.1 Java ecosystem: JDK vs JRE vs JVM

Think of Java in layers:

- **JDK**: used by developers to *create* programs. Includes `javac` (compiler) and other tools.
- **JRE**: used to *run* programs. Includes the JVM and standard libraries.
- **JVM**: the engine that loads classes, verifies bytecode, and executes it.

Rule of thumb. If you want to *write/compile* Java, install the **JDK**. If you only want to *run* Java (rare for students), the **JRE** is enough.

7.2 Bytecode and the compile/run flow

Java is often described as "Write Once, Run Anywhere" because compilation produces **bytecode**, not machine code. The same `.class` file can run on Windows/Linux/macOS *as long as there is a JVM for that OS*.

Flow (mental model):

1. Write: `Hello.java`
2. Compile: `javac Hello.java` → `Hello.class`
3. Run: `java Hello` (JVM loads `Hello.class` and starts `main`)

7.3 Basic program structure: class + `main`

A minimal program has:

- a **class** (container for code),
- the **entry point** method: `public static void main(String[] args)`.

Important rule: if you declare a `public` class, the file name must match it exactly. Example: `public class Hello` must be in `Hello.java`.

7.4 Compile-time vs runtime errors

Compile-time error examples:

- syntax errors (missing semicolon, wrong braces),
- type errors (assigning `String` to `int` without conversion),
- missing symbols (calling a method that doesn't exist).

Runtime error/exception examples:

- `ArithmeticException` (divide by zero),
- `NullPointerException` (calling method on `null`),
- `ArrayIndexOutOfBoundsException` (invalid index).

7.5 Features of Java (what they mean in practice)

- **Platform independent**: bytecode runs on any platform with a JVM.
- **Object-oriented (mostly)**: you model problems using classes/objects (but primitives exist for efficiency).
- **Automatic memory management**: garbage collector reclaims unused objects (no manual `free`).
- **Strong typing**: many errors are caught at compile time (good for large codebases).

- **Rich standard library**: collections, I/O, networking, GUI, database connectivity, etc.
- **Security model**: class loading + bytecode verification help reduce unsafe behavior.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

8 Running Project Connection

Use Java Overview + Program Structure + Features to make the first text-based version of Campus Resource Manager easier to reason about before the full object model arrives.

- Use Campus Resource Manager as the running case study, or map the same idea to your own project domain if you are using another example.
- Ask where today's idea should live: model, service, DAO, UI, or team workflow.
- Use the lecture example as a smaller version of the same design choice you will make in the capstone.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

9 Common Mistakes and Confusions

- Using JDK, JRE, and JVM as interchangeable terms.
- Forgetting that a public class name must match the file name.
- Mixing compile-time errors with runtime failures.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

10 Worked Example: Compile & Run + Observe Errors

A minimal program

```java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

How to run (terminal)

- Compile: `javac Hello.java`
- Run: `java Hello`

Runtime exception example

```java
public class DivideDemo {
    public static void main(String[] args) {
        int a = 10;
        int b = 0;
        System.out.println(a / b); // ArithmeticException at runtime
    }
}
```

Compile-time error example (explain in class)

If you write `System.out.println("Hi")` without a semicolon, `javac` will stop and show an error. This is a compile-time error: the program never runs.

11 Demo / Observation Task

Use this block as a short reproducible demo or observation task.

- Reproduce the lecture's main example around *Java ecosystem: JDK vs JRE vs JVM* once without looking at the solution first.
- Change one input, rule, or design choice in Campus Resource Manager and predict the result before checking it.
- Record one success case, one edge case, and one failure case so the idea becomes observable instead of purely theoretical.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

12 Practice Check (with brief solutions)

1. **JDK/JRE/JVM:** Which one contains `javac`? **Answer:** JDK.
2. **File naming:** What should the file name be if you write `public class Student`? **Answer:** `Student.java`.
3. **Error type:** Is "cannot find symbol" a compile-time or runtime error? **Answer:** compile-time.

13 Self-Study Practice (no solutions)

1. Write `ArgsDemo` that prints all command line arguments with index numbers.
2. Create 2 small programs: one that fails at compile time, one that fails at runtime. Write a 2-line explanation for each.
3. Install the JDK and run `java -version` and `javac -version`. Note the output.

14 Further Study

- Revisit the prerequisite references if any recall bullet still feels weak.
- Rework the lecture example with one changed assumption, input, or design choice.
- Read the textbook and documentation references listed in the source line for a second explanation of the same topic.
- Apply the same idea once inside Campus Resource Manager so that the concept moves from theory to design practice.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

15 Exit Questions (with sample answers)

Q1. Why does Java use bytecode instead of compiling directly to machine code?

Sample answer: Bytecode makes the compiled program portable. The same `.class` can run on different operating systems because the JVM translates/executes bytecode for the current platform.

Q2. What is the difference between a compile-time error and a runtime exception?

Sample answer: Compile-time errors are detected by `javac` before execution; runtime exceptions occur while the program is running after compilation succeeds.

Q3. Why can one .class file run on multiple operating systems?

Sample answer: Because Java compiles to platform-neutral bytecode and each platform provides its own JVM to execute it.